

DSA0502 – Query Processing for Data Science

CO2 AT3 – DEBUGGING TEST LABORATORY REPORT

Course Faculty: Dr. B .SHYAMALA GOWRI

Reg. No.: 192324164

Student Name: Sweety Roselin A

Date: 30/07/2026

Q1. Debug the SQLite Program

Given Program:

```
import sqlite3
conn = sqlite3.connect(college.db)
cursor = conn.cursor
cursor.execute("""
CREATE TABLE Student(
id INTEGER PRIMARY KEY
name TEXT,
marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES(1*, 'John', 90)")
conn.close()
```

Errors Identified & Technical Analysis:

Identified Error	Error Reason / Technical Cause
college.db is not enclosed in quotes	Database name must be passed as a valid string object.
conn.cursor is missing ()	cursor is a method and requires parentheses conn.cursor() to instantiate object.
Missing comma after PRIMARY KEY	Causes invalid SQL syntax error in CREATE TABLE definition.
INSERT statement contains 1*	1* is an invalid literal expression/syntax in SQL value tuples.
conn.commit() is missing	Transactions are not persisted permanently into disk storage without commit.

Cursor is not explicitly closed	Fails to release database lock/memory resources after execution.
CREATE TABLE lacks IF NOT EXISTS	Executing program multiple times raises OperationalError (table exists).

Correct Program:

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
    id INTEGER PRIMARY KEY,
    name TEXT,
    marks INTEGER
)
""")

cursor.execute("INSERT INTO Student VALUES(1, 'John', 90)")
conn.commit()
cursor.close()
conn.close()
```

Purpose of the Corrections:

- Enclosed "college.db" in quotes because the database target path/name must be a valid string.
- Added () to conn.cursor() to properly initialize and instantiate the cursor object.
- Inserted the missing comma after PRIMARY KEY to satisfy ANSI SQL standard DDL syntax.
- Used IF NOT EXISTS to ensure idempotent execution without raising OperationalError on re-runs.
- Added conn.commit() to write transaction logs permanently from RAM memory to storage.
- Closed cursor object explicitly to prevent memory leaks and release database locks.

Q2. Debug the CRUD Program

Given Code Snippet:

```
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchone():
    print(row)
```

Errors Identified:

- fetchone() retrieves only a single record tuple, not the full result set.
- Iterating over fetchone() iterates across individual column values within one row rather than processing all records.

- UPDATE Student SET marks=95 without a WHERE clause accidentally modifies every student record.
- DELETE FROM Student without a WHERE clause completely truncates/empties the table.
- conn.commit() is missing, preventing DML updates/deletions from taking effect permanently.
- Cursor and connection handles are left open.

Correct Program:

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

# 1. Read (Select All)
cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()
for row in rows:
    print(row)

# 2. Update with Predicate
cursor.execute("UPDATE Student SET marks=95 WHERE id=1")

# 3. Delete with Predicate
cursor.execute("DELETE FROM Student WHERE id=2")

conn.commit()
cursor.close()
conn.close()
```

Why Every Record Was Modified or Deleted (Theoretical Analysis):

In SQL relational algebra, DML operations (UPDATE / DELETE) apply to all rows in the target table unless restricted by a filtering predicate. Without a WHERE clause (e.g., WHERE id=1), the engine performs a full table scan and applies the state modification to every single tuple in the table, leading to catastrophic unintended data corruption or complete table erasure.

Q3. Debug the MySQL Connection Program

Given Code:

```
connection=mysql.connector.connect(host="localhost",user="root"password="root",database="college")
```

Errors Identified:

- Missing comma separation between user="root" and password="root" keyword arguments.
- Poor PEP-8 code formatting makes syntax inspection and maintenance difficult.
- Cursor allocation is absent, and connection handles are not closed.

Correct Program:

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="college"
)

cursor = connection.cursor()
cursor.execute("SELECT * FROM student")

for row in cursor:
    print(row)

cursor.close()
connection.close()
```

Why the Original Program Failed: The missing comma between keyword parameters raises a `SyntaxError: invalid syntax` during Python source compilation, preventing script execution entirely.

Q4. Debug the PostgreSQL Program

Given Code Snippet:

```
cursor.execute(
    "INSERT INTO Student VALUES(%s,%s,%s)",
    (1, "Ram")
)
```

Errors Identified:

- Parameter Count Mismatch: Query defines 3 positional placeholders (%s,%s,%s) but parameter tuple contains only 2 elements (1, 'Ram').
- Missing mandatory column value (marks) required by table schema definition.
- Transaction commit (`conn.commit()`) is missing.
- Connection and cursor handles are not closed properly.

Correct Program:

```
import psycopg2

conn = psycopg2.connect(
    dbname="college",
    user="postgres",
    password="admin"
)
```

```

cursor = conn.cursor()

# 3 placeholders mapped exactly to 3 tuple elements
cursor.execute(
    "INSERT INTO Student VALUES(%s, %s, %s)",
    (1, "Ram", 90)
)

conn.commit()

cursor.execute("SELECT * FROM Student")
print(cursor.fetchone())

cursor.close()
conn.close()

```

Effect of Parameter Mismatch: A parameter count mismatch raises a `psycopg2.ProgrammingError` at runtime because the driver's query formatter expects exactly 3 binding elements to substitute into the query template.

Q5. Multiple SQL Errors Analysis

Wrong SQL Statement	Corrected SQL Statement	SQL Category
CREATE TABLE Student(id INTEGER,name TEXT marks INTEGER)	CREATE TABLE Student(id INTEGER, name TEXT, marks INTEGER);	DDL
INSERT Student VALUES(...)	INSERT INTO Student VALUES(1, 'Anu', 89);	DML
SELECT name marks FROM Student;	SELECT name, marks FROM Student;	DQL
UPDATE Student marks=90 WHERE id=1;	UPDATE Student SET marks=90 WHERE id=1;	DML
DELETE Student WHERE id=2;	DELETE FROM Student WHERE id=2;	DML

Q6. Debug the Transaction Program

Errors Identified:

- Logic Flaw: Invoking `conn.rollback()` immediately followed by `conn.commit()` invalidates transaction state control.
- After `rollback()`, all pending uncommitted changes are discarded; executing `commit()` immediately after is redundant and logically contradictory.
- Lack of exception handling context (try-except-finally blocks) to manage transaction safety.

Correct Program:

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

try:
    cursor.execute("UPDATE Student SET marks=80 WHERE id=2")
    conn.commit() # Commit only upon successful execution
except Exception as e:
    conn.rollback() # Rollback transaction on failure
    print(f"Transaction Aborted & Rolled Back: {e}")
finally:
    cursor.close()
    conn.close()
```

Difference Between commit() and rollback():

- commit(): Permanently saves all transaction modifications executed since the last commit/rollback into the database storage files, making changes visible to all concurrent sessions.
- rollback(): Cancels and reverts all uncommitted modifications back to the last committed savepoint state, releasing locks and restoring data integrity.

Q7. Debug the Parameterized Query

Given Code:

```
id = 2
cursor.execute("SELECT * FROM Student WHERE id=" + id)
```

Errors Identified:

- TypeError: Python cannot concatenate string literals ("... WHERE id=") with integer variables (id = 2).
- Severe Security Vulnerability: Dynamic string concatenation exposes the application to SQL Injection (SQLi) attacks.

Correct Program:

```
id = 2

# Using parameterized query placeholder '?' and parameter tuple (id,)
cursor.execute(
    "SELECT * FROM Student WHERE id=?",
    (id,)
)

print(cursor.fetchall())
```

Why Is String Concatenation Unsafe? (SQL Injection Vulnerability):

If user input is constructed via concatenation and receives input like '1 OR 1=1', the database parses the

payload as raw executable SQL logic rather than literal value data: `SELECT * FROM Student WHERE id=1 OR 1=1`. This bypasses authentication or extracts unauthorized records. Parameterized queries enforce strict separation of code and data, sanitizing parameters safely.

Q8. Database Fetch Program

Errors Identified:

- `IndexError` Danger: Unchecked indexing `rows[20]` crashes if `fetchall()` returns fewer than 21 tuples.
- `ProgrammingError`: Attempting to execute SQL statements on a closed cursor object.

Correct Program:

```
cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()

# Boundary Guard / Boundary Check
if len(rows) > 20:
    print(rows[20])
else:
    print("Dataset contains fewer than 21 records.")

# Re-querying while cursor is active
cursor.execute("SELECT * FROM Student")
print(cursor.fetchall())

cursor.close()
```

Q9. Integrated Debugging

Errors Identified:

- Unchecked Table Creation: Missing `IF NOT EXISTS` clause.
- Primary Key Conflict: Attempting to insert duplicate primary key `id=1` raises `IntegrityError`.
- Schema Mismatch: Column name typo (mark instead of marks).
- Entity Reference Error: Table name mismatch (Students instead of Student).
- Missing `commit()` transaction state saving.
- Unclosed cursor handles.

Correct Program:

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
```

```

        id INTEGER PRIMARY KEY,
        name TEXT,
        marks INTEGER
    )
    """

# INSERT OR REPLACE handles primary key conflict gracefully
cursor.execute("INSERT OR REPLACE INTO Student VALUES(1, 'Asha', 90)")
cursor.execute("UPDATE Student SET marks=95 WHERE id=1")
cursor.execute("SELECT name, marks FROM Student")
print(cursor.fetchall())

conn.commit()
cursor.close()
conn.close()

```

Possible Database Exceptions Mapping:

Database Error Scenario	Raised Python Exception
Duplicate primary key insertion	<code>sqlite3.IntegrityError</code>
Invalid column name in query (e.g., mark)	<code>sqlite3.OperationalError</code>
Invalid table name in query (e.g., Students)	<code>sqlite3.OperationalError</code>
Table creation when table already exists (without IF NOT EXISTS)	<code>sqlite3.OperationalError</code>

Q10. Complete Database Application Debugging

Errors Identified:

- Missing commit(): DML modifications (INSERT/UPDATE/DELETE) are lost upon session termination.
- Incorrect Fetch Method: fetchone() returns only a single tuple instead of all table rows.
- Unrestricted UPDATE: UPDATE statement lacks a WHERE clause, updating all database records.
- Syntax Error in DELETE: Usage of python equivalence operator (==) inside SQL query string.
- Cursor handle is left unclosed.

Correct Program:

```

import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

# 1. Insert Record
cursor.execute("INSERT INTO Student VALUES(3, 'Kiran', 90)")

```



```

# 2. Read All Records
cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()
for row in rows:
    print(row)

# 3. Targeted Update
cursor.execute("UPDATE Student SET marks=95 WHERE id=3")

# 4. Targeted Delete
cursor.execute("DELETE FROM Student WHERE id=3")

# 5. Commit and Release
conn.commit()
cursor.close()
conn.close()

```

Benefits of the Applied Corrections:

- conn.commit() ensures persistent storage of transaction operations into college.db.
- fetchall() ensures full dataset retrieval and iteration across all records.
- WHERE id=3 predicate prevents unintended bulk updates/deletions across unrelated records.
- Explicit cursor.close() releases database file locks and prevents resource exhaustion.
- Proper transactional control maintains ACID compliance across database operations.

Predicted Program Output Execution Trace:

Execution Output Trace:

```

--- Initial Student Table State ---
(1, 'John', 90)
(2, 'Anu', 89)

--- Console Output After Read Operation (SELECT * FROM Student) ---
(1, 'John', 90)
(2, 'Anu', 89)
(3, 'Kiran', 90)

--- Final Database Table State (After UPDATE and DELETE of ID=3) ---
(1, 'John', 90)
(2, 'Anu', 89)

```